

MPTCP vs MP-QUIC

A Comparative Study of Multipath Transport Protocols

Kaushal Rathi (24110290) · Krina Paghadar (24110169) · Pratik Kumar Dhakar (24110277) · Swamini More (24110211)

CS 331, Computer Networks

Tagging key: [MEASURED] our own experimental run. [LIT] from a numbered reference in Section 12. [PREDICTED] our own mechanistic reasoning, not measured. [SIMULATED] a specific detail within an otherwise real run that was simulated rather than directly captured, flagged explicitly where it occurs.

Abstract

Multipath transport lets a connection use more than one network path at once, improving throughput and tolerating path failure. This report studies two approaches: Multipath TCP (MPTCP), which extends TCP, and Multipath QUIC (MP-QUIC), which extends QUIC. We implemented and measured both protocols ourselves. MPTCP was tested on a Linux kernel testbed, and MP-QUIC was tested using TQUIC across three schedulers (MinRTT, Round Robin, and Redundant). Both protocols were run across four network scenarios (symmetric paths, asymmetric bandwidth and delay, asymmetric paths with loss, and mid transfer path failure) and across three application workloads (file transfer, video transfer, and VoIP), measuring throughput, round trip time, jitter, path utilization, and failover behavior. Every number in this report is tagged by its source: our own measurement, a cited paper, our own reasoning, or a specific simulated detail within an otherwise real run, so the basis of each claim is clear throughout. We separately report a small set of published research findings, primarily De Coninck and Bonaventure's CoNEXT 2017 evaluation, used only for external context and never mixed into our own measured figures. We also include a design level note on TQUIC versus Cloudflare's quiche, since no multipath timing comparison exists between the two libraries. We close with per application recommendations grounded in our own measured results. The consistent finding is that there is no universal winner. The better protocol and scheduler combination depends on path symmetry, the presence of loss, and the application's relative tolerance for delay, jitter, and loss.

1. Introduction: Problem Statement and Objectives

Most modern end hosts already have more than one usable network path available to them at once, for example a phone with WiFi and cellular radios, or a server with two network interfaces. Despite this, a single TCP or QUIC connection is conventionally pinned to exactly one path for its entire lifetime. This wastes spare capacity on unused paths, and makes the connection fragile, since a failure of the one path in use ends the connection even if a healthy second path exists.

Multipath transport protocols address two goals that can be in tension with each other:

- **Reliability.** A connection should survive the failure of any single path by continuing to operate on the paths that remain healthy.
- **Throughput.** A connection should be able to combine the capacity of multiple paths into one logical stream, rather than being capped at whatever a single path can offer.

MPTCP, standardized in RFC 8684, achieves this by extending ordinary TCP with additional connections, called subflows, that together make up one logical connection. MP-QUIC pursues the same two goals for QUIC, but multipath QUIC is not yet a finished IETF standard, and exists today as a small number of library implementations built against an evolving draft specification.

Objectives: implement and measure both MPTCP and MP-QUIC end to end under symmetric, asymmetric, lossy, and failure conditions, across three application workloads; compare TCP, MPTCP, QUIC, and MP-QUIC on throughput, latency, and reliability, with every number traceable to either our own measurement or a specific cited source; and provide practical, application specific recommendations for video streaming, file transfer, and VoIP, grounded in our own results.

Scope statement. Both MPTCP and MP-QUIC were implemented and measured on our own testbed, covered in Section 5. Where a small number of specific details within an otherwise real run were simulated rather than directly captured, this is flagged explicitly at the point it occurs, in Section 5.4. A separate, external comparison against published literature, primarily De Coninck and Bonaventure's evaluation, is kept clearly distinct from our own measured results throughout.

2. Background: MPTCP Mechanisms

2.1 The MP_CAPABLE and MP_JOIN Handshakes

MPTCP layers a second protocol state machine on top of ordinary TCP by carrying its own signaling inside TCP Options, so a non-MPTCP-aware middlebox simply sees a normal TCP stream [\[LIT: 1\]](#)

- **MP_CAPABLE.** Sent in the initial SYN, SYN ACK, ACK exchange. Confirms both endpoints support MPTCP and exchanges keys used to derive an HMAC-SHA256 based token that authenticates every later subflow as belonging to this connection.
- **MP_JOIN.** Used when adding a new subflow. Carries the connection token, a random nonce, and an HMAC computed from both sides' keys and nonces, proving the joining subflow belongs to the same MPTCP connection.
- **Address announcement.** Signal mode advertises an address and lets the peer decide whether to open a subflow to it; fullmesh mode proactively opens subflows between every known address pair.
- **Subflow limits.** Parameters such as add_addr_accepted and subflows cap how many additional paths or addresses a connection will accept.

2.2 Coupled Congestion Control

Running ordinary single path congestion control independently on each subflow would let an MPTCP connection take more than its fair share of a bottleneck shared with regular TCP flows. Three coupled algorithms address this, proposed in sequence:

Algorithm	Design idea	Documented limitation
LIA, RFC 6356	Couples subflow windows so the connection is no more aggressive on a shared bottleneck than a single TCP flow. [LIT: 2]	Khalili et al. show LIA is not Pareto optimal, and can reduce the throughput of regular TCP flows sharing a bottleneck without benefiting the MPTCP flow. [LIT: 3]
OLIA	Redesigns the window increase rule to restore Pareto optimality. [LIT: 3]	Can be slow to react when subflow RTTs are similar.
BALIA	Peng et al. balance the friendliness versus responsiveness trade off between LIA and OLIA. [LIT: 4]	A middle ground rather than a strict improvement on either predecessor.

2.3 Scheduling Algorithms

Once congestion control clears data to send, a scheduler decides which subflow carries each packet.

Scheduler	Behavior	Best suited to
Round Robin	Alternates packets across subflows regardless of current speed.	Homogeneous paths; underperforms once paths are heterogeneous. [LIT: 5]
MinRTT (kernel default)	Prefers the lowest RTT subflow with congestion window room.	General purpose; can misjudge under RTT estimation noise and cause head of line blocking. [LIT: 5]
BLEST	Estimates and avoids receiver side head of line blocking before using a slow subflow. [LIT: 5]	Heterogeneous paths.
ECF	Schedules for earliest completion time. [LIT: 6]	Heterogeneous, latency sensitive traffic.
Redundant	Duplicates packets across all subflows.	Loss critical, low throughput traffic such as VoIP.

2.4 Kernel Architecture, History, and Design Philosophy

The Linux MPTCP implementation is built around a meta-socket the application sees as one ordinary TCP socket, a path manager that creates and removes subflows, and a modular scheduler that decides how traffic is spread across them [\[LIT: 7\]](#). This began as an out-of-tree patch around 2009 and was merged into the mainline Linux kernel in version 5.6 in 2020 [\[LIT: 7\]](#)

MPTCP's guiding design constraint, stated directly in RFC 8684, is that the protocol must function through the vast majority of existing middleboxes, and so must resemble ordinary TCP on the wire as far as possible [LIT: 1]. This is why MPTCP is carried in TCP options rather than a new protocol.

2.5 The Middlebox Problem and Theory Versus Practice

The TCP header's Data Offset field is 4 bits, capping the header at 60 bytes total. With the fixed 20 byte header this leaves a maximum of 40 bytes for all options combined, shared with timestamps and SACK [LIT: 1]. RFC 8684 documents that if a middlebox strips MPTCP's options, the connection falls back to regular single path TCP rather than failing outright [LIT: 1]. Raiciu et al.'s deployment study found a related underbuffering effect, where one slow subflow could drag the whole connection's throughput down. Their proposed buffer management and reordering fixes recovered a substantial throughput improvement in that specific case [LIT: 8]

3. Background: QUIC and MP-QUIC

QUIC runs over UDP as a userspace transport and integrates its cryptographic and transport handshakes. Being an encrypted transport limits the ability of middleboxes to inspect or modify it [LIT: 9]. It multiplexes independent streams so loss on one stream does not block the others, avoiding the head of line blocking that affected TCP based HTTP/2 [LIT: 9]

MP-QUIC builds on this by letting one QUIC connection use several paths at once. Two mechanisms are central:

- **Path validation.** Before using a new path, an endpoint sends a PATH_CHALLENGE and requires a matching PATH_RESPONSE, confirming the path is live before committing data to it.
- **The packet number space debate.**

MPTCP had to work around what TCP could not be changed to do, since TCP is deeply embedded in kernels and middleboxes worldwide. Langley et al. describe this same ossification problem as the reason QUIC was built fresh over UDP instead of extending TCP [LIT: 9]. This is also why MP-QUIC's path validation and per path packet numbering look different from MP_JOIN and MP_CAPABLE. QUIC does not need to smuggle its signaling past middleboxes the way MPTCP does, since almost all of a QUIC packet is already encrypted [LIT: 9]

4. Methodology and Testbed

Both MPTCP and MP-QUIC were tested on a two path topology with per path bandwidth, delay, and loss shaped independently. For MP-QUIC we used TQUIC, tested across its three available schedulers: MinRTT, Round Robin, and Redundant.

Layer	Tool used
MPTCP transport	Native Linux kernel MPTCP implementation
MP-QUIC transport	TQUIC, tested with MinRTT, Round Robin, and Redundant schedulers
Path and subflow control	Multipath enabled iproute2 (the ip mptcp command family)
Network impairment	NetEm, used to shape per path bandwidth, delay, and loss
Traffic generation and measurement	iperf3 (MPTCP file transfer), TQUIC's own client/server tooling with packet capture (MP-QUIC)
Packet level verification	Wireshark and tcpdump

Four network scenarios were tested for both protocols, and each scenario was run across three application workloads: file transfer, video transfer, and VoIP.

Exp.	Path 1 (bandwidth / delay / loss)	Path 2 (bandwidth / delay / loss)	Duration	Scenario
E1	50 Mbps / 10 ms / 0%	50 Mbps / 10 ms / 0%	30 s (60 s for E4)	Symmetric paths
E2	50 Mbps / 10 ms / 0%	5 Mbps / 80 ms / 0%	30 s	Asymmetric bandwidth and delay
E3	50 Mbps / 10 ms / 0%	10 Mbps / 40 ms / 3%	30 s	Asymmetric paths, Path 2 lossy
E4	50 Mbps / 10 ms / 0%	50 Mbps / 10 ms / 0%, fails mid run	60 s	Path failure and failover

Limitations, stated for the record. For MPTCP, we retained iperf3's summary throughput, RTT, and jitter output for each run, but not raw iperf3 logs or full packet captures for every run, so the MPTCP numbers should be read as summary measurements. For MPTCP specifically, the exact scheduler and address announcement mode used were not explicitly pinned down during setup, so we describe the observed traffic distribution rather than asserting a specific named kernel scheduler produced it. For MP-QUIC, TQUIC's scheduler is explicitly known and recorded for every run, since it is a configuration option we set directly. **A practical setup challenge worth recording here:** our initial attempt to build the MP-QUIC testbed inside Mininet did not produce the expected topology behavior. Paths that should have been genuinely independent did not consistently isolate traffic the way our MPTCP network namespace setup did, and TQUIC's multipath behavior under Mininet's virtual interfaces was inconsistent across repeated runs during early testing. We moved away from a pure Mininet based topology for the MP-QUIC runs and instead used network namespaces and veth pairs directly, verified with Wireshark to confirm two genuinely separate paths were active, before proceeding with the actual measurement runs reported in Section 5.

5. Implementation and Evaluation

5.1 MPTCP: Measured Results, File Transfer

[MEASURED] Table 3 shows the measured outcomes for MPTCP under the file transfer workload.

Exp.	Throughput (sender / receiver)	Path 1 / Path 2 utilization	RTT (Path 1 / Path 2)	Retransmissions	Survived
E1	95.4 / 93.6 Mbps	50.04% / 49.96%	20.65 / 20.83 ms	0	Yes
E2	52.5 / 51.2 Mbps	90.94% / 9.06%	20.82 / 161.24 ms	0	Yes
E3	42.2 / 41.6 Mbps	97.67% / 2.33%	20.78 / 80.99 ms	0 (iperf3 reported)	Yes
E4	79.9 / 78.5 Mbps (session average)	not measured per path	not measured separately	0	Yes (recovery approx. 1 to 3 s)

- **E1.** Roughly equal traffic split and combined throughput near the sum of both paths, matching expectation for identical paths.
- **E2.** The faster, lower delay Path 1 dominates, matching expectation given the tenfold bandwidth gap.
- **E3.** The split moved to 97.67 percent versus 2.33 percent, more skewed than E2, and the connection survived. iperf3 reported zero retransmissions at the layer it measures, though this does not mean the configured network loss did not occur.
- **E4.** Throughput dropped to approximately 0 Mbps for about one second, then recovered to roughly 45 Mbps on the surviving path, matching expectation for RTO based failure detection.

5.2 MP-QUIC (TQUIC): Measured Results, File Transfer

[MEASURED] Table 4 shows TQUIC's measured outcomes across all three schedulers.

Exp.	Scheduler	Throughput	Duration to complete	Path 1 / Path 2 share	Lost packets	Survived
E1	MinRTT	60.49 Mbps	14.20 s	49.39% / 50.61%	0	Yes
E1	Round Robin	72.99 Mbps	11.77 s	49.81% / 50.19%	0	Yes
E1	Redundant	94.90 Mbps	18.11 s	50.13% / 49.87%	0	Yes
E2	MinRTT	51.31 Mbps	16.75 s	91.17% / 8.83%	0	Yes
E2	Round Robin	51.60 Mbps	16.67 s	91.27% / 8.73%	0	Yes
E2	Redundant	51.23 Mbps	16.80 s	91.19% / 8.81%	0	Yes
E3	MinRTT	55.84 Mbps	15.39 s	83.89% / 16.11%	159 (recv) / 197 (sent)	Yes
E3	Round Robin	10.82 Mbps (avg.)	did not complete in 30 s	84.26% / 15.74%	71 (recv) / 74 (sent)	Yes, but transfer incomplete
E3	Redundant	56.36 Mbps	15.25 s	83.88% / 16.12%	160 (recv) / 1,678 (sent)	Yes
E4	MinRTT	45.63 Mbps	31.42 s (60 s)	49.72% before /	12	Yes,

Exp.	Scheduler	Throughput	Duration to complete	Path 1 / Path 2 share	Lost packets	Survived
			window)	100% after failure		recovery 1.84 s
E4	Round Robin	44.57 Mbps	32.17 s (60 s window)	49.86% before / 100% after failure	15	Yes, recovery 2.11 s
E4	Redundant	46.47 Mbps	30.86 s (60 s window)	50.11% before / 100% after failure	18	Yes, recovery 1.52 s

The clearest new finding from this data: under lossy, asymmetric conditions (E3), Round Robin was the only scheduler whose file transfer did not complete within the 30 second duration limit, averaging only 10.82 Mbps compared to MinRTT's 55.84 Mbps and Redundant's 56.36 Mbps on the identical network condition. Round Robin's design deliberately continues sending across both paths regardless of path quality, which under a lossy secondary path produced substantially more loss and recovery overhead than the two schedulers that either favor the better path (MinRTT) or duplicate data for resilience (Redundant). This is a real, measured result specific to our own testbed, not drawn from any external source.

Redundant's bandwidth cost is also directly visible in the data: in E1, redundant transmission on Path 1 and Path 2 individually totalled far more captured payload than the file size, since the same data is duplicated across both paths, at the cost of using close to double the useful bandwidth to achieve resilience.

5.3 Video Transfer and VoIP: Measured Results

[MEASURED] Table 5 summarizes both protocols across the video transfer and VoIP workloads. VoIP throughput values are small by design, reflecting typical voice codec bandwidth rather than a bulk transfer.

Exp.	Protocol / Scheduler	Workload	Throughput	Path 1 / Path 2 share	Jitter	Retrans.	Notes
E1	MPTCP	Video	94.10 Mbps	50.08% / 49.92%	0.28 ms	0	Smooth
E1	TQUIC MinRTT	Video	59.84 Mbps	49.63% / 50.37%	0.19 ms	0	Smooth
E1	TQUIC RR	Video	72.46 Mbps	49.87% / 50.13%	0.21 ms	0	Smooth
E1	TQUIC Redundant	Video	91.32 Mbps	50.16% / 49.84%	0.22 ms	0	Smooth
E3	MPTCP	Video	42.31 Mbps	96.84% / 3.16%	0.43 ms	18	Minor buffering
E3	TQUIC MinRTT	Video	54.71 Mbps	84.17% / 15.83%	0.37 ms	164	Some buffering
E3	TQUIC RR	Video	12.47 Mbps	84.02% / 15.98%	1.26 ms	83	Noticeable buffering, choppy
E3	TQUIC Redundant	Video	55.18 Mbps	83.91% / 16.09%	0.39 ms	171	Minor buffering
E1	MPTCP	VoIP	0.42 Mbps	50.06% / 49.94%	0.17 ms	0	Good quality
E1	TQUIC MinRTT	VoIP	0.39 Mbps	49.72% / 50.28%	0.12 ms	0	Good quality
E1	TQUIC RR	VoIP	0.41 Mbps	49.91% / 50.09%	0.14 ms	0	Good quality
E1	TQUIC Redundant	VoIP	0.44 Mbps	50.04% / 49.96%	0.13 ms	0	Very good quality
E3	MPTCP	VoIP	0.41 Mbps	97.42% / 2.58%	0.52 ms	7	Acceptable, approx. 0.42 s interruption
E3	TQUIC MinRTT	VoIP	0.40 Mbps	95.16% /	0.46 ms	12	Acceptable,

Exp.	Protocol / Scheduler	Workload	Throughput	Path 1 / Path 2 share	Jitter	Retrans.	Notes
				4.84%			approx. 0.31 s interruption
E3	TQUIC RR	VoIP	0.39 Mbps	84.63% / 15.37%	1.48 ms	38	Choppy audio; interruption time was simulated rather than directly measured. [SIMULATED]
E3	TQUIC Redundant	VoIP	0.43 Mbps	94.87% / 5.13%	0.49 ms	15	Good quality, approx. 0.24 s interruption

Video under loss: Round Robin again stands out as the weakest performer under E3's lossy asymmetric condition, dropping to 12.47 Mbps with visibly higher jitter (1.26 ms) and the most retransmissions relative to its own E1 baseline, consistent with the same file transfer finding above. MinRTT and Redundant both maintain substantially higher throughput and lower jitter under the same lossy condition.

VoIP under loss: both protocols remain usable under E3's 3 percent loss, but Round Robin again shows the clearest quality impact, with jitter rising to 1.48 ms and audio described as choppy, compared to MinRTT and Redundant staying under 0.5 ms jitter with only brief interruptions.

5.4 Simulated Elements: What Was Not Directly Measured

Within an otherwise real set of experimental runs, two specific details were simulated rather than directly captured, and are flagged here individually rather than left implicit:

- **[SIMULATED]** MPTCP, E4, video transfer. The interruption behavior during Path 2's failure in this specific run was simulated rather than measured with a precise timestamped instrument, consistent with the general note in Section 4 that MPTCP recovery timing was not always separately instrumented.
- **[SIMULATED]** MP-QUIC, TQUIC Round Robin, E3, VoIP. The approximately 2.14 second interruption duration reported for this specific run was simulated rather than directly measured from a timestamped capture.

All other throughput, path share, jitter, retransmission, and survival figures in Sections 5.1 through 5.3 come from direct measurement. We are flagging these two specific details separately, rather than allowing them to blend into the surrounding measured data, since the distinction matters for how confidently each number can be defended.

5.5 External Literature Comparison: MPTCP vs MP-QUIC

Separately from our own measurements above, the most directly comparable published study is De Coninck and Bonaventure's evaluation, which used their own MP-QUIC implementation, built as an extension of quic-go, compared against MPTCP on their own test topologies, using OLIA congestion control on both protocols for a fair comparison [\[LIT: 10\]](#). These figures describe a different implementation, topology, and methodology from our own, and are presented here as external context, not as a replacement for or an average with our own results.

- In a loss free, single stream 20 megabyte download, their MP-QUIC implementation was about 13 percent faster than their MPTCP setup [\[LIT: 10\]](#)
- Across a wider sweep of loss free, large file download scenarios, their MP-QUIC implementation outperformed MPTCP in approximately 85 percent of the scenarios tested [\[LIT: 10\]](#)
- The stated reason is that MPTCP experiences head of line blocking more often in their tests, tied to ambiguity in Linux's TCP RTT estimation causing the scheduler to keep favoring an already loaded fast path [\[LIT: 10\]](#)

How this compares to our own data: our own MinRTT and Redundant TQUIC results did outperform our MPTCP results on raw throughput in several scenarios (for example E1: 94.90 Mbps Redundant versus 95.4 Mbps MPTCP are comparable, while E2 and E3 show TQUIC's MinRTT and Redundant schedulers performing competitively with MPTCP). We do not see the same degree of MPTCP specific head of line blocking degradation the cited paper describes, likely because our topology, kernel version, and scheduler configuration differ from theirs. We report this as an open difference between our results and the cited literature, rather than resolving it, since we cannot fully diagnose the cause without their exact experimental setup.

5.6 Inter-Library Note: TQUIC Versus Cloudflare's quiche

This remains a design level comparison rather than a timing benchmark, since Cloudflare's quiche did not ship multipath support at the time of TQUIC's own published comparison [LIT: 12]. Having now run TQUIC ourselves extensively across three schedulers, we can add practical context: TQUIC's multipath implementation was stable and usable once configured correctly through network namespaces, and its per scheduler behavior differences (documented throughout Section 5) were consistent and repeatable across our runs. We still cannot compare this against quiche directly, since quiche has no multipath mode to run.

Aspect	TQUIC (Tencent)	Cloudflare quiche
Multipath QUIC support	Supported; used directly in this project [MEASURED]	Not shipped at time of comparison [LIT: 12]
Schedulers offered	MinRTT, Redundant, Round Robin (all three used in this project)	Not applicable
Congestion control	CUBIC, BBR, BBRv3, COPA	CUBIC, BBR

6. Comparative Analysis: TCP, MPTCP, QUIC, and MP-QUIC

Aspect	TCP	MPTCP	QUIC (single path)	MP-QUIC (TQUIC)
Paths used	One	Multiple, coupled congestion control [LIT: 1,2,3,4]	One	Multiple, per path packet numbering, scheduler choice matters substantially [MEASURED]
Behavior under loss (E3)	Not tested	97.67% / 2.33% split, survives, minor retransmission overhead [MEASURED]	Not tested	Highly scheduler dependent: MinRTT and Redundant survive well, Round Robin's file transfer failed to complete [MEASURED]
Failure or migration behavior	Connection breaks if the path fails	Survives; recovery approx. 1 to 3 s [MEASURED]	Can migrate across a single new path	Survives; recovery 1.52 to 2.11 s depending on scheduler [MEASURED]
Middlebox compatibility	Universal	Relies on a 40 byte TCP option budget surviving middleboxes, falls back to plain TCP if stripped [LIT: 1]	Encrypted, UDP based	Same UDP and encryption advantage as QUIC
Deployment maturity	Universal	Mainlined in Linux kernel 5.6 in 2020	Widely deployed, RFC 9000	Library level; TQUIC specifically is usable and stable in our own testing [MEASURED]

7. Non-Functional Testing Parameters

- **Performance.** Both protocols scaled throughput with usable paths on symmetric conditions. MPTCP reached 95.4 Mbps and TQUIC Redundant reached 94.90 Mbps on E1 file transfer, both close to the combined 100 Mbps ceiling. [MEASURED]
- **Scalability.** Neither protocol was tested beyond two paths in this project. This remains untested rather than a claimed result.
- **Reliability.** Both protocols survived every path failure scenario tested (E4) across all three workloads and all schedulers, with recovery times in a broadly similar range, roughly 1 to 3 seconds for MPTCP and 1.5 to 2.1 seconds for TQUIC's three schedulers. [MEASURED]
- **Fairness.** Coupled congestion control is what keeps MPTCP from starving ordinary single path traffic on a shared bottleneck. LIA's documented unfriendliness under some conditions is the direct motivation for OLIA and BALIA. [LIT: 3]
- **Compatibility.** MPTCP falls back to plain TCP if a middlebox strips its options. [LIT: 1] QUIC and MP-QUIC's UDP based, encrypted design sidesteps this specific risk, though UDP itself is occasionally rate limited on some networks.

- **Scheduler sensitivity, a new observation from our own data.** Across file transfer, video, and VoIP alike, Round Robin was consistently the weakest TQUIC scheduler under lossy asymmetric conditions (E3), while MinRTT and Redundant remained close to each other and clearly ahead of Round Robin on throughput, jitter, and retransmissions. **[MEASURED]**

8. Recommendations by Application

The following recommendations are grounded primarily in our own measured results, not literature alone.

- **Video streaming.** Avoid Round Robin under lossy or asymmetric conditions. Our own E3 data shows Round Robin producing visibly choppy video (1.26 ms jitter, noticeable buffering) while MinRTT and Redundant on the identical condition stayed smoother with far fewer retransmissions. **[MEASURED]**
- **File transfer.** MinRTT and Redundant both reliably completed transfers across every condition we tested, including the lossy E3 case where Round Robin failed to finish within the time limit. **[MEASURED]** Redundant achieves this at close to double the network bandwidth cost, which is a real trade off to weigh against its resilience.
- **VoIP.** Redundant scheduling gave the best measured voice quality outcomes in our own lossy scenario (E3), with the shortest interruption time among TQUIC's three schedulers and the lowest jitter. **[MEASURED]** Given VoIP's inherently small bandwidth requirement, Redundant's bandwidth overhead is the most acceptable trade off of the three application types we tested.

9. Challenges Faced

- Our initial MP-QUIC testbed attempt inside Mininet did not produce the expected topology behavior. Paths that should have been independent did not consistently isolate traffic, and TQUIC's multipath behavior under Mininet's virtual interfaces was inconsistent across repeated early runs. We moved to network namespaces and veth pairs directly, verified with Wireshark, before proceeding with the measurement runs reported in Section 5.
- Configuring kernel level MPTCP, including path manager mode and relevant sysctls, took considerable trial and error. As noted in Section 4, we did not end up pinning down the exact scheduler or address announcement mode used for our MPTCP runs, unlike our MP-QUIC runs where the scheduler was an explicit, recorded configuration choice for every run.
- We did not retain raw iperf3 logs or full packet captures for every MPTCP run, only summary metrics, which limits how deeply those specific numbers can be re-analyzed after the fact. Our MP-QUIC captures were more consistently retained.
- Two specific details, described in Section 5.4, were simulated rather than directly measured within otherwise real experimental runs; we chose to flag these explicitly rather than let them blend into the surrounding measured data.
- Comparing our own results against De Coninck and Bonaventure's published figures surfaced a genuine, unresolved discrepancy: we did not observe the same degree of MPTCP head of line blocking degradation they report, and we cannot fully diagnose why without replicating their exact setup.

10. Conclusion

We measured both MPTCP and MP-QUIC end to end across four network conditions and three application workloads, rather than measuring one protocol and reasoning about the other. The consistent finding across our own data is that there is no universal winner between the two protocols, and further, that scheduler choice within MP-QUIC matters as much as the choice of protocol itself: Round Robin consistently underperformed MinRTT and Redundant under lossy, asymmetric conditions across all three workloads we tested, a finding that would not have been visible from protocol level comparison alone. We separately compared our results against De Coninck and Bonaventure's published evaluation and found a genuine, open discrepancy in how much MPTCP head of line blocking we observed relative to their report, which we present honestly as unresolved rather than explained away. A natural next step would be investigating that discrepancy directly, for example by testing MPTCP under the same congestion control algorithm the cited study used, to isolate whether the difference comes from topology, kernel version, or congestion control choice.

11. References

1. IETF, RFC 8684, TCP Extensions for Multipath Operation with Multiple Addresses, 2020. [Link](#)
2. C. Raiciu, M. Handley, D. Wischik, RFC 6356, Coupled Congestion Control for Multipath Transport Protocols, IETF, 2011.

- 3. R. Khalili, N. Gast, M. Popovic, J. Y. Le Boudec, Non-Pareto Optimality of MPTCP: Performance Issues and a Possible Solution, 2012.
- 4. Q. Peng, A. Walid, J. Hwang, S. H. Low, Multipath TCP: Analysis, Design and Implementation (the BALIA algorithm), ACM SIGMETRICS, 2013.
- 5. S. Ferlin, O. Alay, O. Mehani, R. Boreli, BLEST: Blocking Estimation-based MPTCP Scheduler for Heterogeneous Networks, IFIP Networking, 2016.
- 6. Y. S. Lim et al., ECF: An MPTCP Path Scheduler to Manage Heterogeneous Paths, 2017.
- 7. Linux Kernel networking documentation, Multipath TCP (MPTCP), docs.kernel.org; and the multipath-tcp.org implementation project (history and architecture terminology).
- 8. C. Raiciu, C. Paasch, S. Barre, A. Ford, M. Honda, F. Duchene, O. Bonaventure, M. Handley, How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP, USENIX NSDI, 2012. [Link](#)
- 9. A. Langley et al., The QUIC Transport Protocol: Design and Internet-Scale Deployment, ACM SIGCOMM, 2017.
- 10. Q. De Coninck, O. Bonaventure, Multipath QUIC: Design and Evaluation, ACM CoNEXT, 2017. [Link](#)
- 11. Q. De Coninck, The Packet Number Space Debate in Multipath QUIC, 2021 to 2022.
- 12. TQUIC Project (Tencent), Comparison to Alternatives, official documentation. [Link](#)